

Week 6 Report: Access Control and Monitoring

1. Overview

I added four guardrails around my Week 5 TechCorp agent: role-based access control, rate limiting, cost enforcement, and an audit log. The guardrail classes are in `access_control_starter.py`, and I integrated them into a copy of my Week 5 `app_starter.py`. Everything lives in `week6/`. The rule I followed is to block a request before it reaches the LLM, not after.

2. The three guardrail classes

- **AccessController** (loads `data/access_control.json`): `can_view_field` and `can_view_document` check role permissions; `redact_response`, `filter_documents`, and `redact_employee_fields` remove data a role cannot see; every decision is written to an audit log. Unknown sensitivity is denied (fail-safe).
- **RateLimiter**: a sliding 60-second window per user, set to 30 queries per minute.
- **CostEnforcer**: a monthly budget per role (engineer `\$100`, manager `\$500`, hr `\$200`, finance `\$500`, executive `\$1000`). I added a `role` parameter to `can_afford_query` so a new user with no spending record still has a budget to check against.

3. Integration with the Week 5 agent

- `query()` now takes `user_id` (for rate and cost tracking) and `user_role` (for access control).
- The rate limit and budget are checked first, so a blocked request never calls the LLM.
- Access control runs at retrieval time: the employee tool redacts sensitive columns and the policy tool drops documents the role cannot read, so the model never sees restricted data. `redact_response` on the final answer is a second layer. Each lookup and query also writes an audit entry.

4. Offline tests

`test_access_control.py` covers every guardrail without calling the LLM, and all pass. One concrete result: an engineer can retrieve 21 of the 74 policy documents (the Public and Internal ones), while a manager can retrieve all 74.

```

> python test_access_control.py
AccessController
  field visibility: PASSED
  document visibility: PASSED
  filter_documents + audit: PASSED
  redact_response: PASSED
  redact_employee_fields: PASSED
RateLimiter
  per-user limit: PASSED
CostEnforcer
  budget tracking: PASSED
Tool-level enforcement (retrieval time)
  employee field redaction by role: PASSED
  document filtering by role: PASSED (engineer 21 vs manager 74 of 74)
Agent guardrails (block before LLM)
INFO:app_starter:Processing query from rl_user (engineer): anything
  rate-limit block: PASSED
INFO:app_starter:Processing query from budget_user (engineer): anything
  budget block: PASSED

All offline access-control tests passed!
~/Documents/Duke/26Summer/AIPI_561/aipi-561/week6 main* week5 > █

```

The original starter self-test also passes:

```

source ../week5/.venv/bin/activate
python access_control_starter.py
Testing AccessController...
  can_view_field: PASSED
  filter_documents: PASSED
  redact_response: PASSED

Testing RateLimiter...
  is_allowed: PASSED

Testing CostEnforcer...
  can_afford_query: PASSED

All tests passed!
~/Documents/Duke/26Summer/AIPI_561/aipi-561/week6 main week5 > █

```

5. Live demo

`demo_access_control.py` is the only script that calls Gemini.

Same salary question, two roles: the engineer gets a redacted answer, HR gets the figure.

```
> python demo_access_control.py
Agent initialized with access control, rate limiting, cost enforcement

=====
[A1 redacted] role=engineer user_id=eng_user
Q: What is Brian Yang's salary?
A: Brian Yang is the VP of Engineering (Executive). However, his salary information is redacted in the system.
  tokens=1527 cost=$0.000155 budget_remaining=$100.00

=====
[A2 full] role=hr user_id=hr_user
Q: What is Brian Yang's salary?
A: Brian Yang's salary is $467,621.
  tokens=1533 cost=$0.000150 budget_remaining=$200.00
```

A normal policy question still works for an engineer:

```
=====
[B happy-path] role=engineer user_id=eng_user
Q: How many PTO days do employees get per year?
A: According to TechCorp's Paid Time Off Policy, the number of PTO days allocated per year depends on the role:

* **Individual Contributors:** 15 days/year
* **Managers:** 20 days/year
* **Directors and Executives:** 25 days/year
  tokens=1564 cost=$0.000168 budget_remaining=$100.00
```

The rate-limit and budget blocks return an error with no LLM call, followed by the tail of the audit log:

```
=====
[C rate-limit] exhausting the per-minute limit for user 'spammer'
Result: {'error': 'Rate limit exceeded', 'remaining_queries': 0}

=====
[D budget] engineer 'broke_user' has already spent their $100 budget
Result: {'error': 'Budget exceeded', 'budget_remaining': 0.0}

=====
Audit log: 86 entries. Last 5:
{'timestamp': '2026-06-18T22:30:40.687823+00:00', 'role': 'engineer', 'resource': 'doc_internal_048', 'field': None, 'allowed': False}
{'timestamp': '2026-06-18T22:30:40.687824+00:00', 'role': 'engineer', 'resource': 'doc_internal_049', 'field': None, 'allowed': False}
{'timestamp': '2026-06-18T22:30:42.200031+00:00', 'role': 'engineer', 'resource': 'query', 'field': None, 'allowed': True}
{'timestamp': '2026-06-18T22:30:42.200243+00:00', 'role': 'engineer', 'resource': 'rate_limit', 'field': None, 'allowed': False}
{'timestamp': '2026-06-18T22:30:42.200287+00:00', 'role': 'engineer', 'resource': 'budget', 'field': None, 'allowed': False}

Agent metrics: {'total_queries': 3, 'total_tokens': 4624, 'total_cost': 0.00047415, 'avg_cost_per_query': 0.00015805}
~/Documents/Duke/26Summer/ATPT_561/aini-561/week6_main* 1m 7s week5 ^
```